



# **P**ROGRAMACIÓN **O**RIENTADA A **O**BJETOS USANDO JAVA

**UPN**  
**UNIVERSIDAD PRIVADA DEL NORTE**

**CARRERA: INGENIERÍA DE SISTEMAS**

**PROFESOR: MARTIN EDUARDO TORRES RODRIGUEZ**

**ALUMNO: PERCY TEOFILO HUAMAN AÑO**

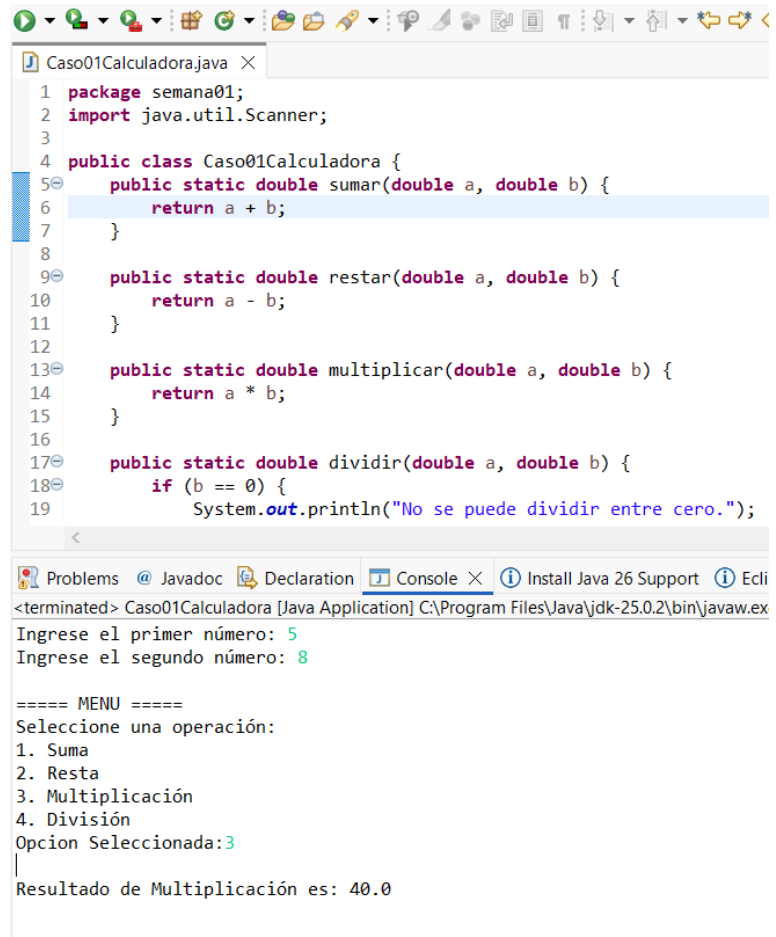
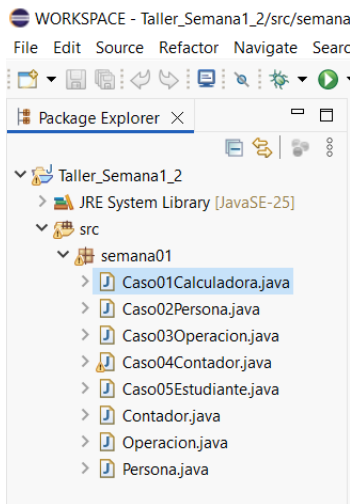
**PRACTICA DE CAMPO SEM-01**

**2026**

# PRACTICA DE CAMPO 01

## CASO 1: Calculadora básica (Condicionales + métodos)

Consigna: Desarrollar un programa que permita: - Ingresar 2 números. - Elegir una operación: suma, resta, multiplicación o división. - Mostrar el resultado. Temas: - Scanner - Condicionales (if / switch) - Métodos



### Caso01Calculadora.java

```
package semana01;  
import java.util.Scanner;
```

```
public class Caso01Calculadora {  
    public static double sumar(double a, double b) {  
        return a + b;  
    }  
  
    public static double restar(double a, double b) {  
        return a - b;  
    }  
  
    public static double multiplicar(double a, double b) {  
        return a * b;  
    }  
    public static double dividir(double a, double b) {  
        if (b == 0) {  
            System.out.println("No se puede dividir entre cero.");  
            return 0;  
        }  
        return a / b;  
    }  
}
```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Ingrese el primer número: "); double num1 = sc.nextDouble();
    System.out.print("Ingrese el segundo número: "); double num2 = sc.nextDouble();
    System.out.println("\n===== MENU =====");
    System.out.print("Seleccione una operación:\n");
    System.out.println("1. Suma");
    System.out.println("2. Resta");
    System.out.println("3. Multiplicación");
    System.out.println("4. División");
    System.out.print("Opcion Seleccionada:"); int opcion = sc.nextInt();

    switch (opcion) {
        case 1:
            System.out.println("\nResultado de Suma es: " + sumar(num1, num2)); break;
        case 2:
            System.out.println("\nResultado de Resta es: " + restar(num1, num2)); break;
        case 3:
            System.out.println("\nResultado de Multiplicación es: " + multiplicar(num1, num2)); break;
        case 4:
            System.out.println("\nResultado de División es: " + dividir(num1, num2)); break;
        default:
            System.out.println("\nOpción inválida.");
    }
    sc.close();
}
}

```

## CASO 2: Clase Persona (POO básico)

Consigna: Crear una clase Persona con: - Atributos: nombre, edad - Método: mostrarDatos() Desde main, crear 2 objetos y mostrar su información. Temas: - Clases y objetos

```

Persona.java ×
1 package semana01;
2
3 public class Persona {
4     String nombre;
5     int edad;
6
7     public void mostrarDatos() {
8         System.out.println("Nombre: " + nombre);
9         System.out.println("Edad: " + edad);
10    }
11 }

```

```

Caso02Persona.java ×
1 package semana01;
2
3 public class Caso02Persona {
4     public static void main(String[] args) {
5         Persona p1 = new Persona();
6         p1.nombre = "Percy";
7         p1.edad = 30;
8
9         Persona p2 = new Persona();
10        p2.nombre = "Maria";
11        p2.edad = 29;
12
13        p1.mostrarDatos();
14        p2.mostrarDatos();
15    }
16 }

```

Problems @ Javadoc Declaration Console × ⓘ In

<terminated> Caso02Persona [Java Application] C:\Program Files\Java

Nombre: Percy  
Edad: 30  
Nombre: Maria  
Edad: 29

### Clase Persona.java

```
package semana01;

public class Persona {
    String nombre;
    int edad;

    public void mostrarDatos() {
        System.out.println("Nombre: " + nombre);
        System.out.println("Edad: " + edad);
    }
}
```

### Caso02Persona.java

```
package semana01;

public class Caso02Persona {
    public static void main(String[] args) {
        Persona p1 = new Persona();
        p1.nombre = "Percy";
        p1.edad = 30;

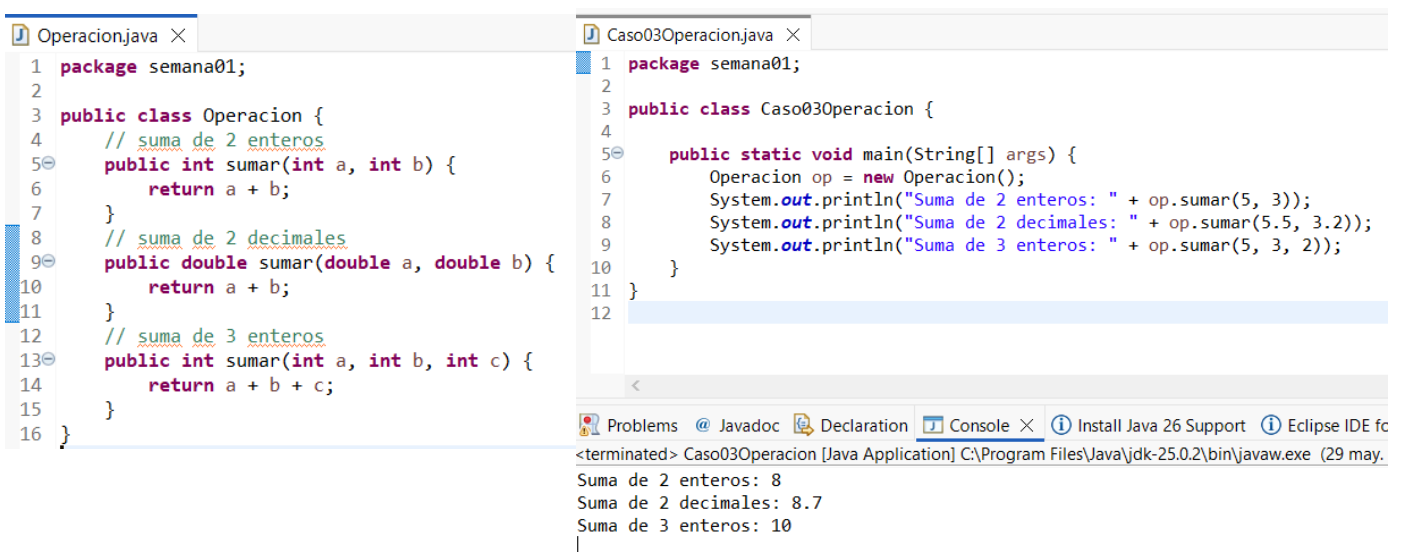
        Persona p2 = new Persona();
        p2.nombre = "Maria";
        p2.edad = 29;

        p1.mostrarDatos();
        p2.mostrarDatos();
    }
}
```

## CASO 3: Sobrecarga de métodos

Consigna: Crear una clase Operacion con métodos: - sumar(int a, int b) - sumar(double a, double b) - sumar(int a, int b, int c) Mostrar resultados desde main.

Temas: - Sobrecarga de métodos - Tipos de datos



```
Operacion.java
1 package semana01;
2
3 public class Operacion {
4     // suma de 2 enteros
5     public int sumar(int a, int b) {
6         return a + b;
7     }
8     // suma de 2 decimales
9     public double sumar(double a, double b) {
10        return a + b;
11    }
12    // suma de 3 enteros
13    public int sumar(int a, int b, int c) {
14        return a + b + c;
15    }
16 }
```

```
Caso03Operacion.java
1 package semana01;
2
3 public class Caso03Operacion {
4
5     public static void main(String[] args) {
6         Operacion op = new Operacion();
7         System.out.println("Suma de 2 enteros: " + op.sumar(5, 3));
8         System.out.println("Suma de 2 decimales: " + op.sumar(5.5, 3.2));
9         System.out.println("Suma de 3 enteros: " + op.sumar(5, 3, 2));
10    }
11 }
12
```

Problems @ Javadoc Declaration Console × Install Java 26 Support Eclipse IDE for

<terminated> Caso03Operacion [Java Application] C:\Program Files\Java\jdk-25.0.2\bin\javaw.exe (29 may.

Suma de 2 enteros: 8  
Suma de 2 decimales: 8.7  
Suma de 3 enteros: 10

### Clase Operación.java

```
package semana01;

public class Operacion {
    // suma de 2 enteros
    public int sumar(int a, int b) {
        return a + b;
    }
    // suma de 2 decimales
    public double sumar(double a, double b) {
        return a + b;
    }
    // suma de 3 enteros
    public int sumar(int a, int b, int c) {
        return a + b + c;
    }
}
```

### Caso03Operacion.java

```
package semana01;

public class Caso03Operacion {

    public static void main(String[] args) {
        Operacion op = new Operacion();
        System.out.println("Suma de 2 enteros: " + op.sumar(5, 3));
        System.out.println("Suma de 2 decimales: " + op.sumar(5.5, 3.2));
        System.out.println("Suma de 3 enteros: " + op.sumar(5, 3, 2));
    }
}
```

## CASO 4: Uso de static

Consigna: Crear una clase Contador: - Variable estática contador - Método que incremente el contador cada vez que se crea un objeto Mostrar cuántos objetos se han creado. Temas: - static - Variables compartidas

```
*Contador.java X
1 package semana01;
2
3 public class Contador {
4
5     // variable estática (compartida por todos los objetos)
6     private static int contador = 0;
7
8     // constructor (se ejecuta al crear un objeto)
9     public Contador() {
10         contador++;
11     }
12
13     // método para obtener el valor del contador
14     public static int getContador() {
15         return contador;
16     }
17 }
```

```
Caso04Contador.java ×
1 package semana01;
2
3 public class Caso04Contador {
4
5     public static void main(String[] args) {
6
7         // creación de objetos
8         Contador c1 = new Contador();
9         Contador c2 = new Contador();
10        Contador c3 = new Contador();
11        Contador c4 = new Contador();
12
13        // mostrar cuántos objetos se crearon
14        System.out.println("Objetos creados: " + Contador.getContador());
15    }
16 }
17

Problems @ Javadoc Declaration Console × Install Java 26 Support Eclipse IDE
minated> Caso04Contador [Java Application] C:\Program Files\Java\jdk-25.0.2\bin\javaw.exe (29 may.
objetos creados: 4
```

### Contador.java

```
package semana01;

public class Contador {

    // variable estática (compartida por todos los objetos)
    private static int contador = 0;

    // constructor (se ejecuta al crear un objeto)
    public Contador() {
        contador++;
    }

    // método para obtener el valor del contador
    public static int getContador() {
        return contador;
    }
}
```

### Clase04Contador.java

```
package semana01;

public class Caso04Contador {

    public static void main(String[] args) {

        // creación de objetos
        Contador c1 = new Contador();
        Contador c2 = new Contador();
        Contador c3 = new Contador();
        Contador c4 = new Contador();

        // mostrar cuántos objetos se crearon
        System.out.println("Objetos creados: " + Contador.getContador());
    }
}
```

## CASO 5: Lista de estudiantes

Consigna: Crear un programa que: - Permita ingresar nombres de estudiantes (usar ArrayList) - Mostrar la lista completa - Manejar error si el usuario ingresa datos inválidos Temas: - ArrayList - Bucles - Manejo básico de errores (try-catch)

```
Caso05Estudiante.java X
1 package semana01;
2
3 import java.util.ArrayList;
4
5
6
7 public class Caso05Estudiante {
8
9     public static void main(String[] args) {
10         Scanner sc = new Scanner(System.in);
11         ArrayList<String> estudiantes = new ArrayList<>();
12         int opcion = 0;
13
14         do {
15             System.out.println("\n===== MENU =====");
16             System.out.println("1. Agregar estudiante");
17             System.out.println("2. Mostrar estudiantes");
18             System.out.println("3. Salir");
19             System.out.print("Seleccione Opción: ");
20
21             try {
22                 opcion = sc.nextInt();
23                 sc.nextLine(); // limpiar buffer
24
25                 switch (opcion) {
26
27                     case 1:
28                         System.out.print("Ingrese nombre del estudiante: ");
29                         String nombre = sc.nextLine();
30                         estudiantes.add(nombre);
31                         System.out.println("Estudiante agregado.");
32                         break;
33
34                     case 2:
35                         System.out.println("\nLista de estudiantes:");
36                         if (estudiantes.isEmpty()) {
37                             System.out.println("No hay estudiantes.");
38                         } else {
39                             for (String est : estudiantes) {
40                                 System.out.println("- " + est);
41                             }
42                         }
43                         break;
44
45                     case 3:
46                         System.out.println("Saliendo...");
47                         break;
48
49                     default:
50                         System.out.println("Opción inválida.");
51                 }
52             } catch (InputMismatchException e) {
53                 System.out.println("Error: debe ingresar un número.");
54                 sc.nextLine(); // limpiar entrada incorrecta
55             }
56
57         } while (opcion != 3);
58
59         sc.close();
60     }
61 }
62 }
```

Problems @ Javadoc Declaration Console X  
<terminated> Caso05Estudiante [Java Application] C:\Program

```
===== MENU =====
1. Agregar estudiante
2. Mostrar estudiantes
3. Salir
Seleccione Opción: 1
Ingrese nombre del estudiante: Percy
Estudiante agregado.

===== MENU =====
1. Agregar estudiante
2. Mostrar estudiantes
3. Salir
Seleccione Opción: 1
Ingrese nombre del estudiante: Juan
Estudiante agregado.

===== MENU =====
1. Agregar estudiante
2. Mostrar estudiantes
3. Salir
Seleccione Opción: 2

Lista de estudiantes:
- Percy
- Juan

===== MENU =====
1. Agregar estudiante
2. Mostrar estudiantes
3. Salir
Seleccione Opción: 3
Saliendo...
```

## Caso05estudiante.java

```
package semana01;

import java.util.ArrayList;
import java.util.InputMismatchException;
import java.util.Scanner;

public class Caso05Estudiante {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<String> estudiantes = new ArrayList<>(); // lista para guardar estudiantes
        int opcion = 0;

        do {
            System.out.println("\n===== MENU =====");
            System.out.println("1. Agregar estudiante");
            System.out.println("2. Mostrar estudiantes");
            System.out.println("3. Salir");
            System.out.print("Seleccione Opción: ");

            try {
                opcion = sc.nextInt();
                sc.nextLine(); // limpiar buffer

                switch (opcion) {
                    case 1:
                        System.out.print("Ingrese nombre del estudiante: ");
                        String nombre = sc.nextLine();
                        estudiantes.add(nombre);
                        System.out.println("Estudiante agregado.");
                        break;
                    case 2:
                        System.out.println("\nLista de estudiantes:");
                        if (estudiantes.isEmpty()) {
                            System.out.println("No hay estudiantes.");
                        } else {
                            for (String est : estudiantes) {
                                System.out.println("- " + est);
                            }
                        }
                        break;
                    case 3:
                        System.out.println("Saliendo...");
                        break;
                    default:
                        System.out.println("Opción inválida.");
                }
            }

            } catch (InputMismatchException e) {
                System.out.println("Error: debe ingresar un número.");
                sc.nextLine(); // limpiar entrada incorrecta
            }

        } while (opcion != 3);
    }
}
```



```
    sc.close();  
  }  
}
```

## PREGUNTAS DE REFLEXIÓN

### ¿Cuál es la diferencia entre un método normal y uno static?

Un método normal pertenece a un objeto y necesita ser llamado mediante una instancia de la clase, mientras que un método static pertenece a la clase y puede ser llamado sin crear un objeto.

### ¿En qué casos usarías sobrecarga de métodos?

La sobrecarga de métodos se usa cuando se necesita realizar la misma operación con diferentes tipos o cantidades de datos, permitiendo reutilizar el mismo nombre de método con diferentes parámetros.

- ✓ Cuando quieres hacer lo mismo, pero con diferentes tipos de datos
- ✓ Cuando el comportamiento es similar, pero cambian los parámetros
- ✓ Para evitar crear muchos nombres de métodos

### Indicaciones Generales

- Trabajar en Eclipse. - Crear un proyecto llamado: Taller\_Semana1\_2. - Cada caso debe estar en una clase diferente.
- Subir el código a GitHub. - Realizar al menos 2 commits.

## PASOS GIT - GITHUB

Abrimos GitBash

Configuramos usuario y email

Creamos repositorio del proyecto \$ git init

```
MINGW64:/d/WORKSPACE/Taller_Semana1_2  
  
T570@DESKTOP-6KJV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2  
$ git config --global user.name "PercyHuaman"  
$ git config --global email.name "noreply.github.com"  
T570@DESKTOP-6KJV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2  
$ git init  
Initialized empty Git repository in D:/WORKSPACE/Taller_Semana1_2/.git/
```

- Adicionamos a área de preparación      \$ git add .
- Se crea primer commit                      \$ git commit -m "Primer commit – Practica del 1 al 5"
- Listamos el commit creado                \$ git log

```

MINGW64:/d/WORKSPACE/Taller_Semana1_2

T570@DESKTOP-6KJV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2
$ git init
Initialized empty Git repository in D:/WORKSPACE/Taller_Semana1_2/.git/

T570@DESKTOP-6KJV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$ git add .

T570@DESKTOP-6KJV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$ git commit -m "Primer Commit - Practica del 1 al 5"
[main (root-commit) 8558fe9] Primer Commit - Practica del 1 al 5
 9 files changed, 199 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 src/semana01/Caso01Calculadora.java
 create mode 100644 src/semana01/Caso02Persona.java
 create mode 100644 src/semana01/Caso03Operacion.java
 create mode 100644 src/semana01/Caso04Contador.java
 create mode 100644 src/semana01/Caso05Estudiante.java
 create mode 100644 src/semana01/Contador.java
 create mode 100644 src/semana01/Operacion.java
 create mode 100644 src/semana01/Persona.java

T570@DESKTOP-6KJV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$ git log
commit 8558fe9cb868c941ddbbbc07e6a33b2f147c0acb8 (HEAD -> main)
Author: PercyHuaman <91505139+PercyHuaman@users.noreply.github.com>
Date: Sat May 30 01:53:39 2026 -0500

    Primer Commit - Practica del 1 al 5

```

## TRABAJANDO CON RAMAS

### BRANCH

```

MINGW64:/d/WORKSPACE/Taller_Semana1_2

T570@DESKTOP-6KJV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$ git branch rama-practicas

T570@DESKTOP-6KJV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$ git switch rama-practicas
Switched to branch 'rama-practicas'

T570@DESKTOP-6KJV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (rama-practicas)
$ git status
On branch rama-practicas
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/semana01/Caso01Calculadora.java

no changes added to commit (use "git add" and/or "git commit -a")

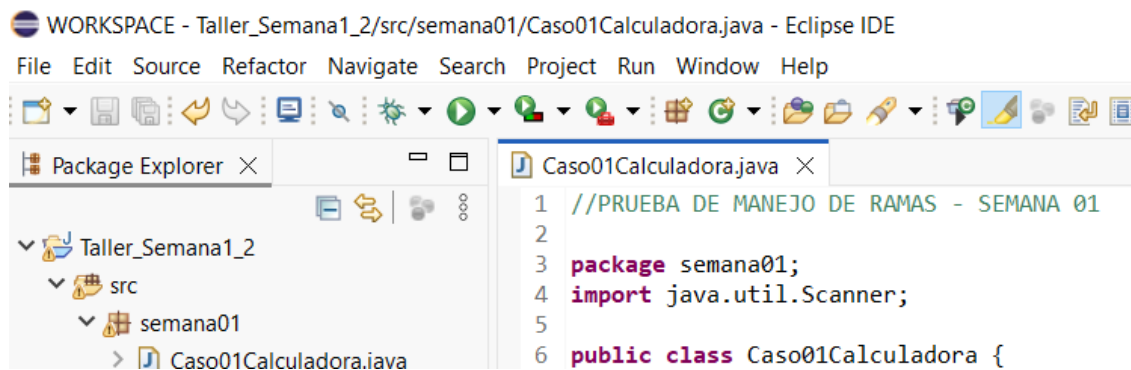
T570@DESKTOP-6KJV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (rama-practicas)
$ git add .

T570@DESKTOP-6KJV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (rama-practicas)
$ git commit -m "Agregar comentario de prueba en rama-practicas"
[rama-practicas 1064c61] Agregar comentario de prueba en rama-practicas
 1 file changed, 2 insertions(+)

```

- Creamos rama                                \$ git branch rama-practicas
- Nos ubicamos en rama creada            \$ git switch rama-practicas

- Ingresamos a eclipse y colocamos un comentario en código de Clase01Calculadora .java



- Verificamos el estado `$ git status`
- Adicionamos a preparación `$ git add .`
- Realizamos el commit `$ git commit -m "Agregar comentario de prueba en rama-practicas"`

Con log verificamos nuestros commits

```

T570@DESKTOP-6KJVG38 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (rama-practicas)
$ git log
commit 1064c610fc22e8bc0ec2c8e549408a1a320166ff (HEAD -> rama-practicas)
Author: PercyHuaman <91505139+PercyHuaman@users.noreply.github.com>
Date: Sat May 30 02:30:52 2026 -0500

    Agregar comentario de prueba en rama-practicas

commit 8558fe9cb868c941ddbcb07e6a33b2f147c0acb8 (main)
Author: PercyHuaman <91505139+PercyHuaman@users.noreply.github.com>
Date: Sat May 30 01:53:39 2026 -0500

    Primer Commit - Practica del 1 al 5

```

Regresamos a main `$ git switch main`

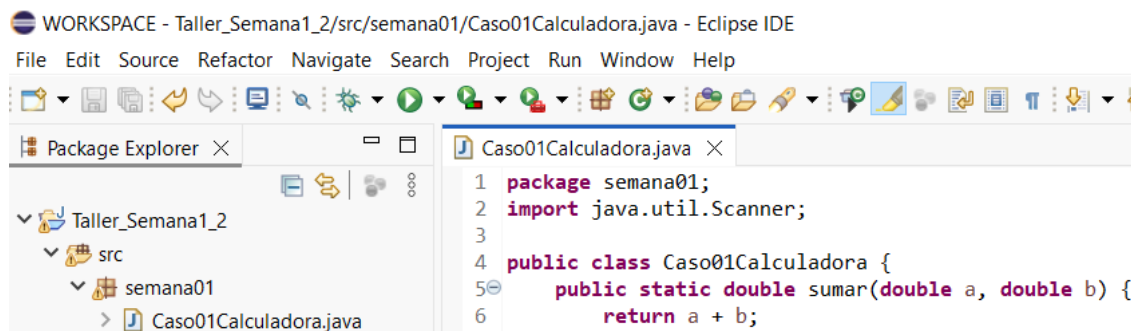
```

T570@DESKTOP-6KJVG38 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (rama-practicas)
$ git switch main
Switched to branch 'main'

T570@DESKTOP-6KJVG38 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$

```

Vamos a eclipse y vemos que el comentario que adicionamos desaparecio, porque el cambio hicimos en la rama



## MERGE

Nos ubicamos en main porque es donde lo vamos a fusionar

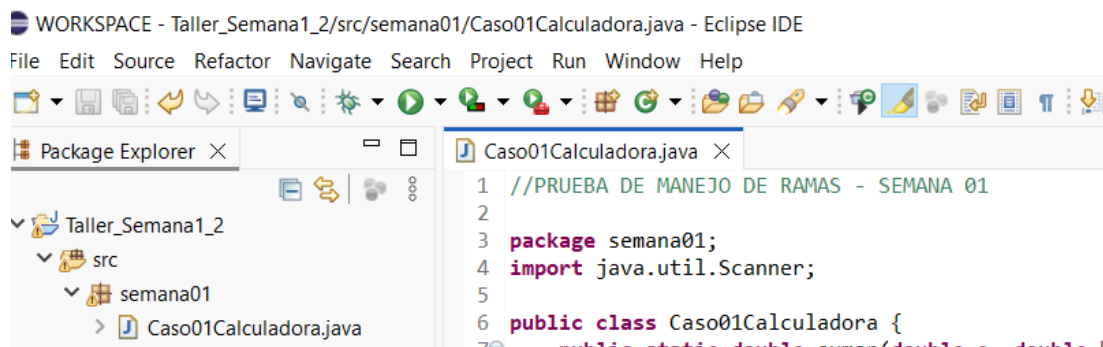
Fusionamos con merge `$ git merge rama-practicas`

```
T570@DESKTOP-6KJVV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (rama-practicas)
$ git switch main
Switched to branch 'main'

T570@DESKTOP-6KJVV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$ git merge rama-practicas
Updating 8558fe9..1064c61
Fast-forward
 src/semana01/Caso01Calculadora.java | 2 ++
 1 file changed, 2 insertions(+)

T570@DESKTOP-6KJVV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$ |
```

Entramos a Eclipse y observamos que apareció el comentario, ahora main esta actualizado



Como ya fusionamos las ramas una buena practica es eliminarlas `$ git branch -d rama-practicas`

```
T570@DESKTOP-6KJVV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$ git branch
* main
  rama-practicas

T570@DESKTOP-6KJVV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$ git branch -d rama-practicas
Deleted branch rama-practicas (was 1064c61).

T570@DESKTOP-6KJVV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$ |
```

## GITHUB

- Creamos repositorio nuevo con el mismo nombre de nuestro proyecto luego en git local
- `$ git remote add origin https://github.com/PercyHuaman/Taller\_Semana1\_2.git ## conecta con github`
- `$ git push -u origin main ### sube todo el código a remoto internet`

```
T570@DESKTOP-6KJVV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$ git remote add origin https://github.com/PercyHuaman/Taller_Semana1_2.git

T570@DESKTOP-6KJVV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$ git push -u origin main
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 4 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (18/18), 3.25 KiB | 1.08 MiB/s, done.
Total 18 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/PercyHuaman/Taller_Semana1_2.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

T570@DESKTOP-6KJVV638 MINGW64 /d/WORKSPACE/Taller_Semana1_2 (main)
$ |
```

Taller\_Semana1\_2 Público

Alfiler

Mirar 0

principal

1 Sucursal

0 etiquetas

Ir al archivo

Agregar archivo

Código

|                                     |                                                              |                         |           |
|-------------------------------------|--------------------------------------------------------------|-------------------------|-----------|
| PercyHuaman                         | Compromiso 3: Agregar guía de pasos de Git finalizada en PDF | 8cca169 · Hace 1 minuto | 3 Commits |
| src/ semana01                       | Agregar comentario de prueba en rama-practicas               | hace 1 hora             |           |
| .gitignore                          | Primer Compromiso - Práctica del 1 al 5                      | hace 1 hora             |           |
| Taller_Semana1_2 - Percy Huaman.pdf | Compromiso 3: Agregar guía de pasos de Git finalizada en PDF | Hace 1 minuto           |           |

Archivos

principal

Ir al archivo

src/semana01

- Caso01Calculadora.java
- Caso02Persona.java
- Caso03Operacion.java
- Caso04Contador.java
- Caso05Estudiante.java
- Contador.java
- Operacion.java
- Persona.java
- .gitignore
- Taller\_Semana1\_2 - Percy Huama...

Taller\_Semana1\_2 / src / semana01 /

PercyHuaman Agregar comentario de prueba en rama-practicas

| Nombre                 | Mensaje del último commit                      |
|------------------------|------------------------------------------------|
| ...                    |                                                |
| Caso01Calculadora.java | Agregar comentario de prueba en rama-practicas |
| Caso02Persona.java     | Primer Compromiso - Práctica del 1 al 5        |
| Caso03Operacion.java   | Primer Compromiso - Práctica del 1 al 5        |
| Caso04Contador.java    | Primer Compromiso - Práctica del 1 al 5        |
| Caso05Estudiante.java  | Primer Compromiso - Práctica del 1 al 5        |
| Contador.java          | Primer Compromiso - Práctica del 1 al 5        |
| Operación.java         | Primer Compromiso - Práctica del 1 al 5        |
| Persona.java           | Primer Compromiso - Práctica del 1 al 5        |